

Master Test Plan

smoltcp v0.12.0

Security & Quality Testing Campaign

TCP/IP Stack Quality Assurance

Test Planning Document

April 2026

Contents

1	Test Plan Overview	2
1.1	Purpose	2
1.2	Scope	2
1.3	Standards and References	2
2	Test Objectives	2
3	Test Environment	3
3.1	Hardware	3
3.2	Software Configurations	3
3.3	Configuration Strategy	3
4	Test Suites	3
4.1	Suite 1: Mutation Adequacy	3
4.2	Suite 2: Input Space Partitioning (UDP)	4
4.3	Suite 3: IPv4 Combinatorial	4
4.4	Suite 4: Security Fuzzing	4
4.5	Suite 5: Conformance Testing	4
4.6	Suite 6: Performance Testing	4
4.7	Suite 7: White-Box Adequacy	5
4.8	Suite 8: Software Safety	5
5	Evaluation Criteria	5
6	Test Execution Schedule	5
7	Test Deliverables	6
8	Risk Assessment	6

1 Test Plan Overview

1.1 Purpose

This Master Test Plan outlines the strategy, environment, resources, schedule, and evaluation criteria for a comprehensive quality assessment of smoltcp v0.12.0, an open-source, event-driven TCP/IP stack written in Rust. The plan coordinates eight distinct testing phases spanning mutation adequacy, functional testing, security analysis, conformance testing, performance evaluation, white-box adequacy, safety assessment, and defect correction.

1.2 Scope

Testing focuses on four high-complexity source modules selected for their critical role in packet processing:

- `src/storage/assembler.rs` — TCP segment reassembly with gap tracking
- `src/wire/ipv4.rs` — IPv4 header parsing, validation, and emission
- `src/wire/udp.rs` — UDP datagram parsing and checksum verification
- `src/wire/tcp.rs` — TCP segment parsing, flag extraction, and option handling

1.3 Standards and References

- RFC 791 — Internet Protocol (IPv4)
- RFC 768 — User Datagram Protocol (UDP)
- RFC 793 — Transmission Control Protocol (TCP)
- RFC 2460 — Internet Protocol, Version 6 (IPv6)
- IEEE 802.15.4 — Low-Rate Wireless Personal Area Networks
- IETF RFC 6282 — 6LoWPAN Header Compression

2 Test Objectives

The overall testing objectives are:

1. **Evaluate test adequacy** via mutation analysis on four complex modules and quantify kill ratios per module and per operator class.
2. **Improve test suite** based on surviving non-equivalent mutants by adding targeted assertions.
3. **Perform functional testing** using the category-partition method with combinatorial test generation for UDP and IPv4 parsing.
4. **Conduct security fuzzing** with coverage-guided mutation (libFuzzer) to identify crash-inducing inputs.
5. **Verify conformance** against RFC 791 and RFC 768 header parsing requirements.
6. **Benchmark performance** using loopback throughput measurement and microbenchmarks.
7. **Measure white-box coverage** using LLVM source-based coverage instrumentation.
8. **Assess software safety** through panic-safety testing and unsafe code inventory.
9. **Apply defect corrections** for any discovered specification deviations.

3 Test Environment

3.1 Hardware

All testing is conducted on a single host machine with the following specifications:

- CPU: AMD/Intel x86_64 processor
- RAM: 16 GB minimum
- Storage: SSD with ≥ 50 GB free space for build artifacts

3.2 Software Configurations

Two platform configurations are tested:

Table 1: Test environment configurations.

Attribute	Windows	WSL
Operating System	Windows 10 (64-bit)	Ubuntu 24.04.2 LTS
Shell	PowerShell 7.x	Bash 5.x
Rust Stable	1.91+	1.91+
Rust Nightly	1.96.0-nightly	1.96.0-nightly
C Compiler	MSVC (cl.exe)	GCC/Clang
cargo-fuzz	0.13.1	0.13.1
cargo-llvm-cov	0.8.5	0.8.5
Python	3.12+	3.12+

3.3 Configuration Strategy

Given resource constraints, the following testing strategy is applied:

- Both platforms are tested extensively (“extreme point” approach).
- Core functional tests (mutation, partitioning, conformance, safety, coverage) run on both platforms.
- Fuzzing is Linux-only due to MSVC/libFuzzer incompatibility.
- Performance results are compared across platforms to identify OS-specific bottlenecks.

4 Test Suites

4.1 Suite 1: Mutation Adequacy

- **Objective:** Evaluate test suite effectiveness via first-order mutation analysis.
- **Method:** Custom Python mutant generator applying 7 operator classes.
- **Target files:** 4 source modules, ≥ 300 total mutants.
- **Oracle:** Module-level unit test suite (cargo test -lib).
- **Metrics:** Kill ratio $K/(M - E)$ per file and overall.
- **Deliverables:** mutants.diff, mutation_results_{platform}.csv, per-mutant execution logs.

```
1 python mutate_and_test.py --source src/storage/assembler.rs \  
2   --operators rel,arith,bool,const,bit,assign,shift \  
3   --output-dir logs/ --platform windows
```

Listing 1: Mutation testing invocation.

4.2 Suite 2: Input Space Partitioning (UDP)

- **Objective:** Systematic functional testing of UDP parsing via the category-partition method.
- **Method:** Full combinatorial coverage of 6 input dimensions.
- **Total cases:** 480 test frames.
- **Oracle:** Expected `new_checked()` and `parse()` outcomes derived from RFC 768 and smoltcp behavior.
- **Deliverables:** `udp_cases.csv`, partition suite source code, execution logs.

4.3 Suite 3: IPv4 Combinatorial

- **Objective:** Category-partition testing of IPv4 header parsing.
- **Method:** Combinatorial coverage of version, IHL, total length, fragmentation, and checksum.
- **Total cases:** 216 test frames.
- **Oracle:** Aligned to smoltcp's actual fragmentation and header-length behavior.
- **Deliverables:** `ipv4_cases_{platform}.csv`, JSON summary.

4.4 Suite 4: Security Fuzzing

- **Objective:** Discover crash-inducing inputs via coverage-guided mutation fuzzing.
- **Method:** libFuzzer (via cargo-fuzz) with sanitizer instrumentation.
- **Targets:** 5 fuzz harnesses (`packet_parser`, `tcp_headers`, `dhcp_header`, `ieee802154_header`, `sixlowpan_packet`).
- **Time budget:** 60s initial + 180s extended per target.
- **Oracle:** Process exit code (crash = failure); sanitizer reports for root cause.
- **Deliverables:** Crash artifacts, reproduction logs, coverage statistics.

4.5 Suite 5: Conformance Testing

- **Objective:** Verify adherence to RFC 791 (IPv4) and RFC 768 (UDP).
- **Method:** Crafted packets testing specific header validation rules.
- **Total cases:** 9 hand-crafted conformance assertions.
- **Oracle:** RFC-derived expected accept/reject decisions.
- **Deliverables:** `conformance_results.csv`, per-platform logs.

4.6 Suite 6: Performance Testing

- **Objective:** Measure raw throughput and per-operation latency.

- **Methods:** Loopback throughput benchmark (128 MB transfer, 5 runs) and microbenchmarks (5 repetitions).
- **Metrics:** Gbps throughput, ns/iteration latency.
- **Deliverables:** CSV metrics per platform, execution logs.

4.7 Suite 7: White-Box Adequacy

- **Objective:** Quantify source code coverage.
- **Tool:** cargo-llvm-cov v0.8.5 (LLVM source-based coverage).
- **Metrics:** Region, function, and line coverage percentages.
- **Deliverables:** Coverage reports, LCOV artifacts, per-module breakdowns.

4.8 Suite 8: Software Safety

- **Objective:** Verify panic-free operation on invalid inputs and audit unsafe code usage.
- **Methods:** Panic-safety test suite; `grep -r "unsafe"` inventory with manual classification.
- **Deliverables:** Safety test logs, `unsafe_inventory.csv`.

5 Evaluation Criteria

Table 2: Evaluation criteria for each test dimension.

Dimension	Criteria
Mutation adequacy	Kill ratio $K/(M - E) \geq 70\%$ per file; overall $\geq 65\%$
Input partitioning	Pairwise coverage achieved; $\geq 95\%$ tests passing
Fuzzing	≥ 6 distinct error signatures documented (or justification if fewer)
Conformance	All tests pass or deviations documented with rationale
Performance	Median throughput and variance reported; cross-platform comparison
White-box adequacy	$\geq 80\%$ line coverage for target modules
Safety	No panics on invalid inputs; all unsafe usage reviewed

6 Test Execution Schedule

Testing follows a sequential single-threaded execution model:

1. **Phase 1 — Infrastructure:** Create test suites, harnesses, and automation scripts.
2. **Phase 2 — Windows execution:** Run all suites sequentially on Windows (PowerShell).
3. **Phase 3 — WSL execution:** Run all suites sequentially on WSL (Bash).
4. **Phase 4 — Enhanced suites:** Execute IPv4 combinatorial, microbenchmarks, enhanced fuzzing, and white-box additions.
5. **Phase 5 — Analysis:** Analyze surviving mutants, compare cross-platform results, identify defects.
6. **Phase 6 — Remediation:** Apply defect corrections, re-run relevant suites to verify fixes.
7. **Phase 7 — Reporting:** Generate final report, test plan, and presentation.

7 Test Deliverables

- **Test automation scripts:** Python runners, PowerShell/Bash wrappers for each suite.
- **Documented test cases:** CSV files with inputs, expected outputs, and actual results.
- **Mutation artifacts:** Unified diff file (`mutants.diff`) and per-mutant CSV results.
- **Test logs:** Per-suite, per-platform execution logs with timestamps.
- **Coverage reports:** LLVM-cov and LCOV formatted coverage data.
- **Incident reports:** Conformance deviations and portability failures.
- **Statistical summaries:** JSON summary files for each enhanced suite.
- **Final report:** Comprehensive write-up with figures and analysis.

8 Risk Assessment

Table 3: Testing risks and mitigations.

Risk	Impact	Mitigation
MSVC/libFuzzer incompatibility	No fuzzing on Windows	Defer to WSL; document as portability gap
Long fuzzing campaigns	Limited time budget per target	Use 60s+180s budgets; prioritize coverage
Equivalent mutants inflate scores	Misleading adequacy ratios	Manual equivalence classification for top survivors
TunTap Linux-only gating	No official example on Windows	Develop custom loopback harness